



**UNIVERSIDAD
DE LA SERENA**
CHILE

Ramificación y Poda (Branch & Bound)

Visualización de Árboles en el TSP

Estudiante: Sara Román Leiva

Asignatura: Diseño y Análisis de Algoritmos

Profesor: Cristian Cubillos

1. Introducción y Objetivos

El Problema del Viajante de Comercio (TSP) es un problema NP-duro de optimización combinatoria. Mientras que el backtracking puro explora el espacio de estados de forma exhaustiva sufriendo una explosión exponencial, la técnica de Ramificación y Poda (Branch & Bound) optimiza esta búsqueda utilizando funciones de acotación (bounding functions) para descartar subárboles completos que no prometen superar a la mejor solución conocida (incumbente).

Este laboratorio implementa Branch & Bound para el TSP con reducción de matrices para calcular la cota inferior (Lower Bound) de cada nodo. El sistema exporta el árbol completo a JSON y Graphviz DOT con metadatos completos por nodo: ID, camino parcial, cota calculada y estado de finalización.

Objetivos de aprendizaje

- Diferenciar empíricamente el impacto de las estrategias de exploración (LIFO y Best-First) en la topología del árbol de búsqueda.
- Implementar un motor de instrumentación que exporte la dinámica de ramificación y poda a JSON estructurado.
- Analizar la sensibilidad de una función de acotación basada en reducción de matrices frente a variaciones controladas.

2. Escenario Experimental

Se trabaja con una instancia asimétrica y no euclidiana de 5 ciudades (A, B, C, D, E). Los costos de tránsito se definen en la siguiente matriz:

Desde / Hacia	A	B	C	D	E
A	∞	14	4	10	20
B	14	∞	7	8	12
C	4	5	∞	16	3
D	11	7	16	∞	2
E	18	10	4	2	∞

Solución óptima: A → C → E → D → B → A, costo total = 30

3. Desarrollo e Instrumentación

3.1 Arquitectura del sistema

El proyecto está compuesto por los siguientes archivos fuente:

Archivo	Descripción
bnb_tsp.py	Motor principal: reducción de matrices, estrategias LIFO y Best-First, exportación JSON/DOT
pregunta_4_2_cota_ingenua.py	Variante con cota ingenua para la Pregunta 4.2
pregunta_4_3_espejismo.py	Análisis del efecto espejismo con $C \rightarrow E=99$ para la Pregunta 4.3
generar_capturas.py	Genera los PNG de cada árbol desde los JSON exportados

3.2 Metadatos exportados por nodo

Cada nodo del árbol exportado contiene los siguientes campos, tal como requiere la guía:

Campo	Descripción
id	Entero único asignado en orden estricto de instanciación
camino	Secuencia de vértices recorridos (ej. $A \rightarrow C \rightarrow E$)
lb	Cota inferior calculada tras reducir la matriz residual
estado	Expandido / Podado por Cota / Podado por Inviabilidad / Solución Completa
color	Codificación cromática del estado (azul / rojo / naranja / verde)
nivel	Profundidad en el árbol (raíz = 0)

3.3 Reducción del nodo raíz

La cota inferior inicial se calcula reduciendo la matriz de costos completa: se resta el mínimo de cada fila y luego el mínimo de cada columna de la matriz resultante.

Paso 1 — Reducción de filas

Fila	Mínimo	Aporte al LB	Operación
A	4	4	Resta 4 a toda la fila A
B	7	7	Resta 7 a toda la fila B
C	3	3	Resta 3 a toda la fila C
D	2	2	Resta 2 a toda la fila D

Fila	Mínimo	Aporte al LB	Operación
E	2	2	Resta 2 a toda la fila E
Subtotal		18	

Paso 2 — Reducción de columnas (sobre la matriz ya reducida)

Columna	Mínimo	Aporte al LB	Observación
A	1	1	Queda un residuo mínimo en col A
B	2	2	Queda un residuo mínimo en col B
C	0	0	Col C ya tiene un 0
D	0	0	Col D ya tiene un 0
E	0	0	Col E ya tiene un 0
Subtotal		3	

LB raíz = $18 + 3 = 21$

Matriz reducida resultante

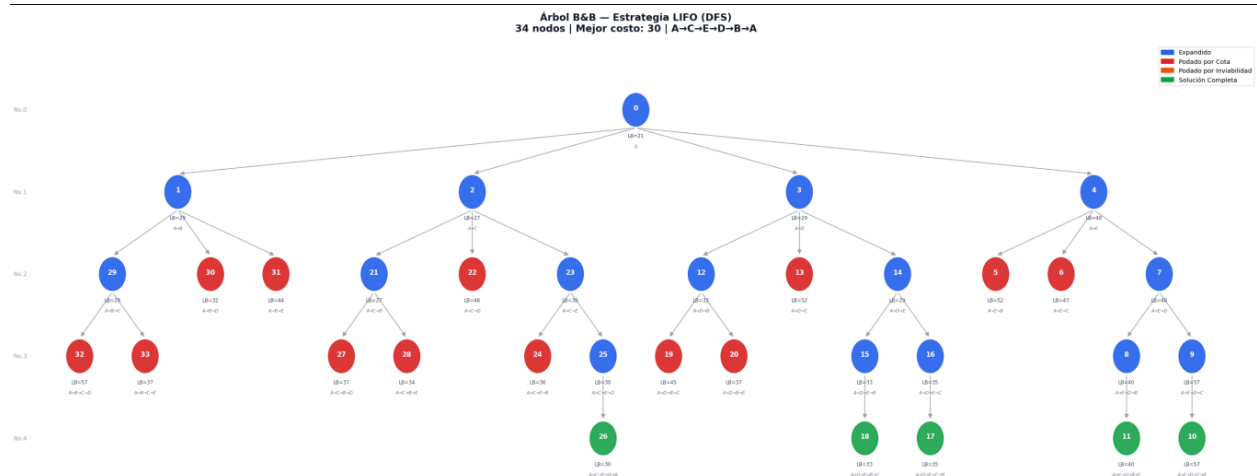
	A	B	C	D	E
A	∞	8	0	6	16
B	6	∞	0	1	5
C	0	0	∞	13	0
D	8	3	14	∞	0
E	15	6	2	0	∞

4. Cuestionario de Evaluación

4.1 Anatomía de la Poda y la Incumbente Temprana

Se ejecutó el algoritmo con dos estrategias de selección distintas. A continuación se presentan los árboles generados por el software y el análisis de los eventos de poda.

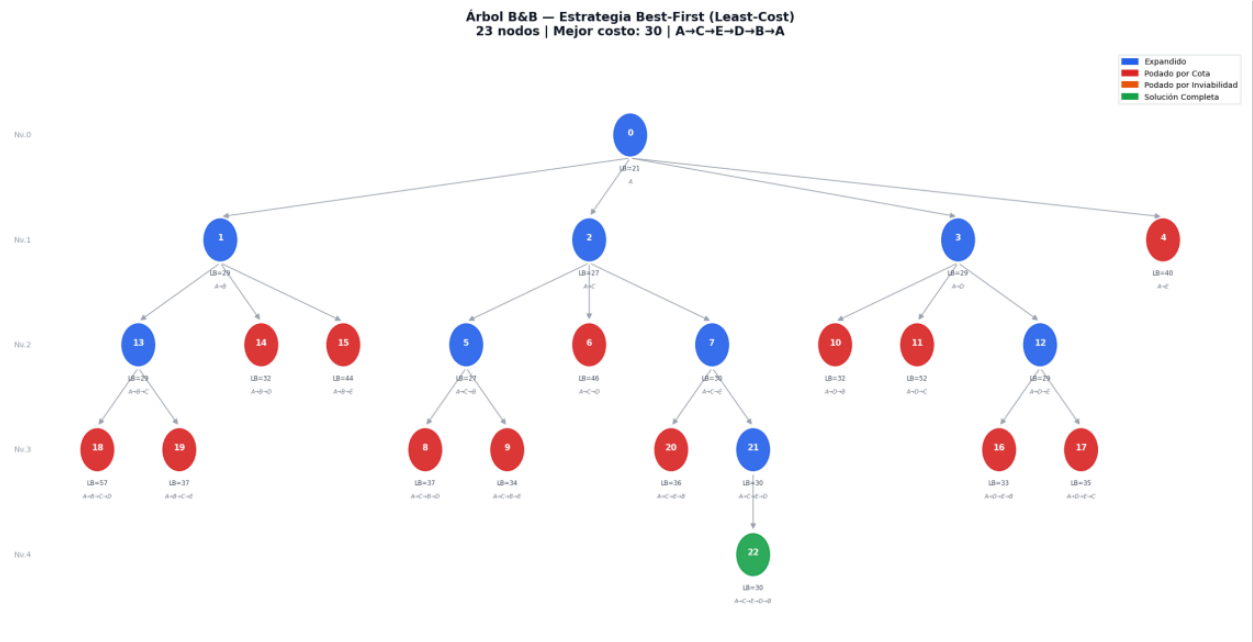
Árbol LIFO (Depth-First) — 34 nodos instanciados



Primera incumbente válida: Nodo ID 10, camino [A→E→D→C→B], costo = 57.

Bajo LIFO el algoritmo apila los hijos y siempre expande el último insertado, descendiendo en profundidad de forma inmediata. La rama A→E fue la última apilada en nivel 1 y por lo tanto la primera en explorarse. Esto llevó rápidamente a una hoja (Nodo 10), pero por un camino costoso: la primera incumbente encontrada tiene costo 57, lejos del óptimo global (30). Este es el comportamiento típico de DFS: llega a soluciones rápido pero sin garantía de calidad.

Árbol Best-First (Least-Cost) — 23 nodos instanciados



Nodo creado pero jamás expandido: Nodo ID 4, camino [A→E], LB = 40.

El nodo 4 fue instanciado en nivel 1 junto con los demás hijos de la raíz. Sin embargo, la estrategia Best-First siempre expande el nodo de menor LB disponible. Para cuando el nodo 4 (LB=40) llegó al frente de la cola de prioridad, el algoritmo ya había encontrado la solución óptima de costo 30. La condición de poda se cumple:

$$LB(\text{nodo } 4) = 40 \geq \text{incumbente} = 30 \rightarrow \text{PODADO} \checkmark$$

Explorar una rama cuyo mejor caso posible (40) ya supera la mejor solución conocida (30) no aporta ninguna mejora. El nodo fue descartado sin generar ningún hijo.

Comparación directa LIFO vs Best-First

Métrica	LIFO (DFS)	Best-First
Nodos instanciados	34	23
Primera incumbente	Nodo ID 10, costo 57	—
Costo final óptimo	30	30
Nodo con solución óptima	Nodo 26	Nodo 22
Reducción de nodos	—	−32% respecto a LIFO

4.2 Sensibilidad Estructural ante Variaciones de la Función de Acotación

Se modificó el código para reemplazar la cota robusta (reducción completa de filas y columnas) por una cota ingenua: suma de los costos mínimos salientes de las ciudades no visitadas, sin penalizar la pérdida de conectividad hamiltoniana ni actualizar columnas.

Árbol con cota ingenua (Best-First) — 18 nodos

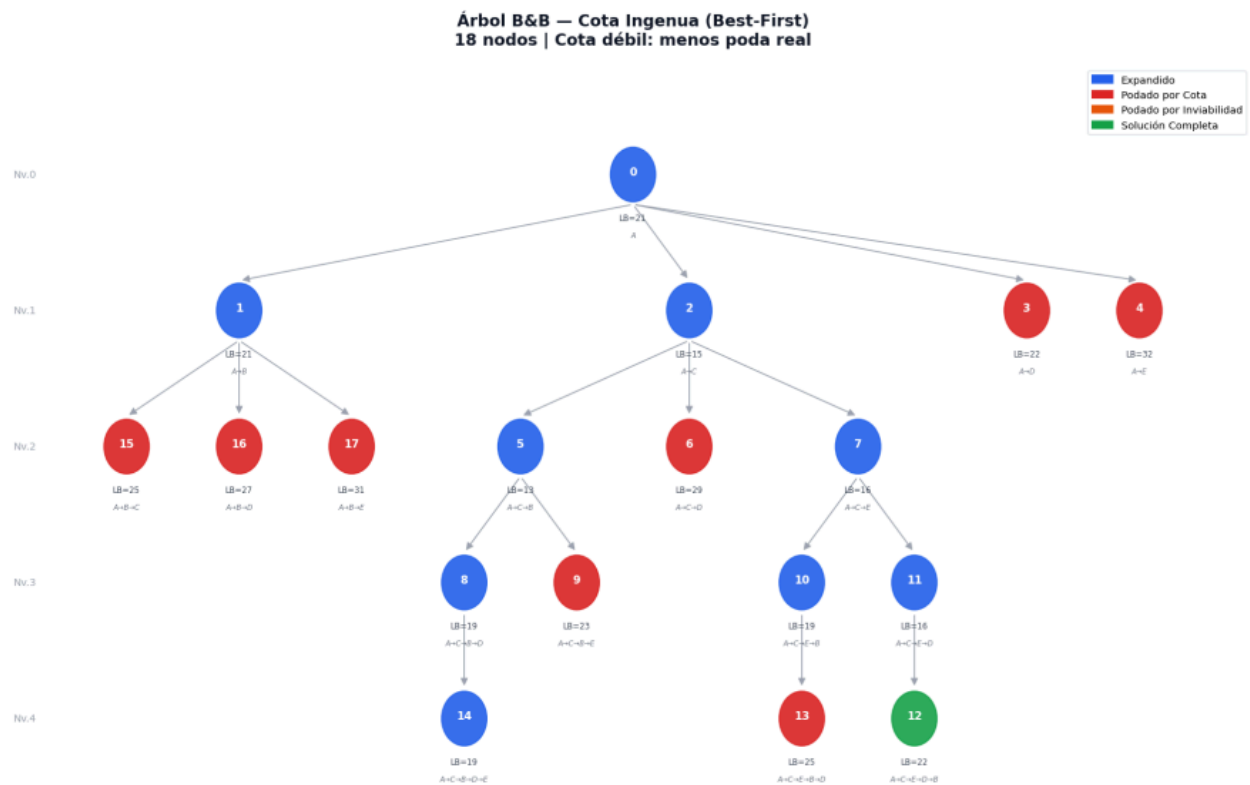


Tabla comparativa de nodos instanciados (Best-First)

Esquema de acotación	Nodos instanciados	Observación
Cota robusta (reducción de matriz)	23	Referencia base
Cota ingenua (suma mínimos salientes)	18	-22% en esta instancia

Nota: la cota ingenua genera menos nodos en esta instancia de 5 ciudades, pero eso no implica que sea mejor. La cota ingenua es sistemáticamente subevaluada respecto a la cota real. En instancias más grandes esto produce retraso en la poda: ramas que deberían descartarse temprano continúan expandiéndose porque su LB ficticio es bajo.

Comparación a nivel 3 — nodo $A \rightarrow C \rightarrow B \rightarrow E$

Método	LB calculado	Cómo se obtiene
Cota robusta	34	LB padre (27) + costo arco $B \rightarrow E$ en mat. reducida (7) + reducción residual (0) = 34
Cota ingenua	23	Costo real acumulado $A \rightarrow C \rightarrow B \rightarrow E$ ($4+5+12=21$) + mínimo saliente de D (2) = 23

La diferencia de 11 puntos ocurre porque la cota ingenua ignora que las columnas ya bloqueadas por aristas previas elevan el costo mínimo necesario para completar el ciclo hamiltoniano. Cuando se elige la arista $B \rightarrow E$, se bloquea la fila B y la columna E; la reducción de la matriz residual captura automáticamente que ahora es más caro conectar los nodos restantes. La cota ingenua descarta este efecto y subvalúa el estado real del sistema.

Fenómeno de retraso en la poda

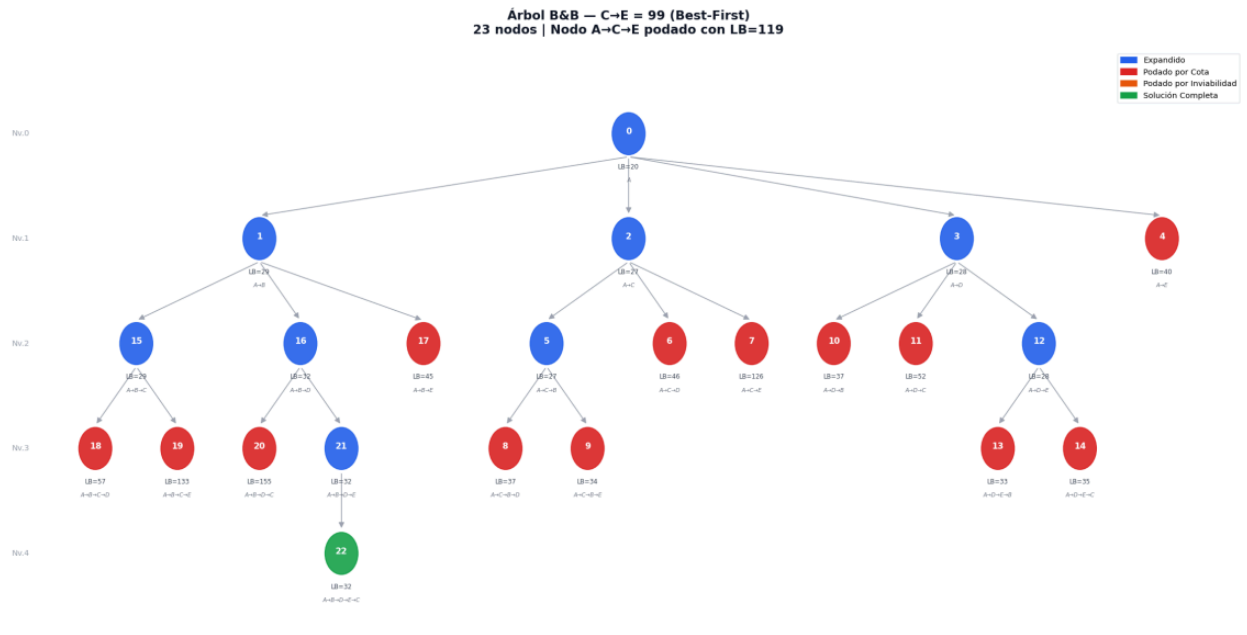
Con cota ingenua, el camino $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E$ llega hasta nivel hoja (nodo 14, LB=19). La cota robusta habría podado este camino en nivel 3 porque la reducción residual de la matriz ya refleja que no quedan columnas baratas disponibles para completar el ciclo.

Este comportamiento no puede predecirse sin instanciar la matriz reducida: el costo adicional para cerrar el ciclo depende de cuáles columnas quedaron bloqueadas, información que es combinatoriamente dependiente del camino exacto recorrido. Ningún análisis de mínimos independientes puede capturar esta interdependencia sin construir la matriz paso a paso.

4.3 El "Efecto Espejismo" y Resiliencia Topológica

Se modificó un único elemento de la matriz: el costo del arco $C \rightarrow E$ se estableció en 99 (valor prohibitivo). El resto de los parámetros permaneció idéntico. Se regeneró el árbol con estrategia Best-First.

Árbol Best-First con $C \rightarrow E = 99$ — 23 nodos



Comparación de los primeros 3 niveles

ID	Camino	LB Original	LB $C \rightarrow E=99$	Estado en árbol modificado
0	A	21	20	Expandido
1	$A \rightarrow B$	29	29	Expandido
2	$A \rightarrow C$	27	27	Expandido
3	$A \rightarrow D$	29	28	Expandido
4	$A \rightarrow E$	40	40	Podado por cota
7	$A \rightarrow C \rightarrow E$	30	119	Podado por cota (LB=119 >> 30)

Los IDs de los primeros nodos se mantienen estables porque el orden de generación es determinístico: el algoritmo siempre expande A primero y genera sus 4 hijos en el mismo orden ($A \rightarrow B$, $A \rightarrow C$, $A \rightarrow D$, $A \rightarrow E$). El cambio en $C \rightarrow E$ no afecta qué nodos se crean primero, solo cuándo se podan. La diferencia se hace drástica a partir del nivel 2: la rama $A \rightarrow C \rightarrow E$, que

antes era el camino hacia la solución óptima (LB=30), ahora tiene LB=119 y es podada en el acto.

Demostración analítica — Reducción del nodo raíz con $C \rightarrow E = 99$

Paso 1 — Reducción de filas:

Fila	Mínimo original	Mínimo $C \rightarrow E=99$	Explicación del cambio
A	4	4	Sin cambio
B	7	7	Sin cambio
C	3	4	+1: pierde $C \rightarrow E=3$, nuevo mínimo es $C \rightarrow A=4$
D	2	2	Sin cambio
E	2	2	Sin cambio
Subtotal filas	18	19	+1

Paso 2 — Reducción de columnas (sobre la matriz ya reducida por filas):

Columna	Mínimo original	Mínimo $C \rightarrow E=99$	Explicación del cambio
A	1	0	-1: el cero de fila C ahora está en $C \rightarrow A$, no en $C \rightarrow E$
B	2	1	-1: el mínimo de col B se reduce
C	0	0	Sin cambio
D	0	0	Sin cambio
E	0	0	El mínimo ahora viene de $D \rightarrow E=2 \rightarrow 0$
Subtotal cols	3	1	-2

LB raíz original = $18 + 3 = 21$ → LB raíz $C \rightarrow E=99$ = $19 + 1 = 20$ (bajó en 1)

Interpretación del efecto espejismo: aunque $C \rightarrow E=99$ hace ese arco prácticamente inviable, el nodo raíz reporta una cota menor que el original. Esto ocurre porque la reorganización de mínimos residuales produce un balance neto negativo: la fila C sube en 1 por perder su mínimo barato ($C \rightarrow E=3$), pero las columnas bajan en 2 porque ese mínimo ya no "ocupa" la columna E. El balance global es -1.

Este es el efecto espejismo: una alteración local costosa puede reducir la cota global porque reorganiza cómo se distribuyen los mínimos en la matriz. El espacio de búsqueda aparenta ser más prometedor en la raíz, aunque internamente hay una ruta ahora completamente bloqueada. La naturaleza global de las cotas locales implica que ningún cambio en un arco es

verdaderamente aislado: toda modificación redistribuye los mínimos de filas y columnas afectando la cota de todo el subárbol.

5. Archivos Generados

El sistema produce los siguientes archivos al ejecutar el motor principal:

Archivo	Descripción
arbol_lifo.json	Árbol LIFO — 34 nodos con metadata completa
arbol_best.json	Árbol Best-First — 23 nodos con metadata completa
arbol_cota_ingenua.json	Árbol con cota ingenua — 18 nodos
arbol_ce99.json	Árbol con $C \rightarrow E=99$ — 23 nodos
arbol_lifo.dot	Formato Graphviz DOT del árbol LIFO
arbol_best.dot	Formato Graphviz DOT del árbol Best-First
arbol_cota_ingenua.dot	Formato Graphviz DOT cota ingenua
arbol_ce99.dot	Formato Graphviz DOT $C \rightarrow E=99$
capturas/captura_lifo.png	Visualización PNG del árbol LIFO
capturas/captura_best.png	Visualización PNG del árbol Best-First
capturas/captura_cota_ingenua.png	Visualización PNG cota ingenua
capturas/captura_ce99.png	Visualización PNG $C \rightarrow E=99$

Para renderizar los archivos DOT con Graphviz:

```
dot -Tpng arbol_best.dot -o arbol_best_graphviz.png
```

6. Repositorio de Código Fuente

Enlace al repositorio GitHub: <https://github.com/SaraRoman23/Lab-28-05-2026.git>

El repositorio contiene el código fuente completo, los archivos JSON exportados, las capturas PNG de los árboles y este informe. Para reproducir todos los resultados:

```
git clone <url-del-repositorio>
cd bnb-tsp
pip3 install matplotlib
python3 bnb_tsp.py
python3 generar_capturas.py
```

Cada archivo de pregunta puede ejecutarse de forma independiente para verificar los resultados específicos de las Preguntas 4.2 y 4.3.